



南京理工大学

7 随机算法

Random Algorithm

suntingkai@njust.edu.cn



随机算法的设计思想

- 允许结果以较小的概率出现错误，以此为代价，获得算法运行时间的大幅度减少。如果一个问题没有有效的确定性算法可以在一个合理的时间给出解答，但是该算法能接受小概率的错误，那么采用随机算法就可以快速地找到这个问题的解。
- 例如：判断表达式 $f(x_1, x_2, \dots, x_n)$ 是否恒等于0，若用解析的方式来判断非常费时。随机算法首先随机生成一个 n 元向量 $(r_1, r_2, \dots, r_n)$ ，若 $f(r_1, r_2, \dots, r_n) \neq 0$ ，则表达式 $f \equiv 0$ 显然不成立；
- 若 $f(r_1, r_2, \dots, r_n) = 0$ ，或者是 $f \equiv 0$ ，或者是选择的 $(r_1, r_2, \dots, r_n)$ 比较凑巧，那么多试验几次，若继续得到 $f(r_1, r_2, \dots, r_n) = 0$ 的结论，就可以得出 $f(r_1, r_2, \dots, r_n) \equiv 0$ 的结论，并且试验的次数越多，出错的概率越小。



随机数发生器

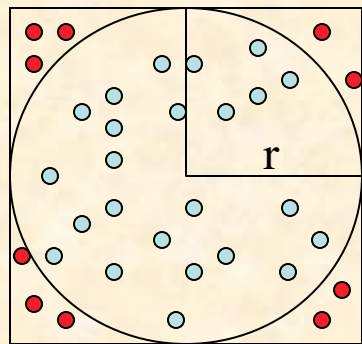
- 随机数在概率算法设计中扮演着十分重要的角色。在现实计算机上无法产生真正的随机数，因此在概率算法中使用的随机数都是一定程度上随机的，即伪随机数。线性同余法是产生伪随机数的最常用的方法。由线性同余法产生的随机序列 $a_0, a_1, \dots, a_n$ 满足

$$\begin{cases} a_0 = d \\ a_n = (ba_{n-1} + c) \bmod m \end{cases} \quad n = 1, 2, \dots$$

- 其中 $b \geq 0$, $c \geq 0$, $d \leq m$ 。d称为该随机序列的种子。如何选取该方法中的常数b、c和m直接关系到所产生的随机序列的随机性能。这是随机性理论研究的内容，已超出本书讨论的范围。从直观上看，m应取得充分大，因此可取m为机器大数，另外应取 $\gcd(m, b) = 1$ ，因此可取b为一素数。

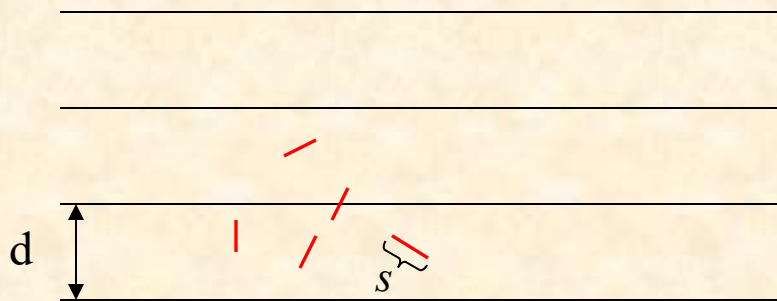


圆周率计算



$$\frac{S_{circle}}{S_{square}} = \frac{\pi \cdot r^2}{4 \cdot r^2} \approx \frac{n_{in_circle}}{n_{in_square}} \quad \pi \approx 4 \cdot \frac{n_{in_circle}}{n_{in_square}}$$

```
public static double darts(int n)
{ // 用随机投点法计算pi值
  int k=0;
  for (int i=1;i <=n;i++) {
    double x=dart.fRandom();
    double y=dart.fRandom();
    if ((x*x+y*y)<=1) k++;
  }
  return 4*k/(double)n;
}
```

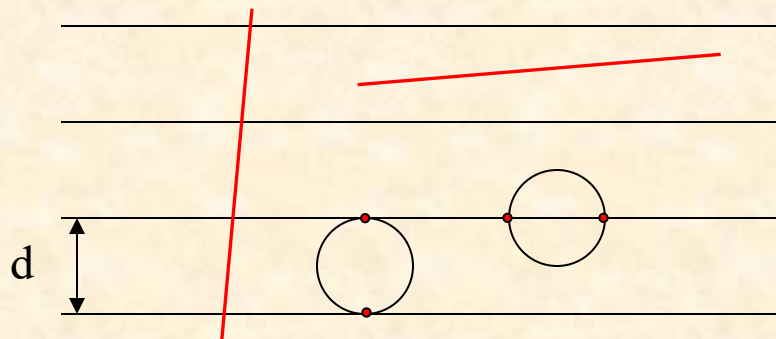


18世纪，法国数学家布丰和勒可莱尔提出的“投针问题”，记载于布丰1777年出版的著作中：“在平面上画有一组间距为 d 的平行线，将一根长度为 s （ $s < d$ ）的针任意掷在这个平面上，求此针与平行线中任一条相交的概率。”

- 1) 取一张白纸，在上面画上许多条间距为 d 的平行线。
- 2) 取一根长度为 s （ $s < d$ ）的针，随机地向画有平行直线的纸上掷 n 次，观察针与直线相交的次数，记为 m
- 3) 计算针与直线相交的概率 p

布丰本人证明了，这个概率是

$$p = \frac{2s}{\pi d} \approx \frac{m}{n}$$



- 一个直径为 d 的圆圈，不管如何投掷，必定是有两个交点。那么投掷 n ，共有 $2n$ 个交点。
- 把圆圈拉直，变成一条长为 πd 的线段。显然，这样的线段扔下时与平行线相交的情形要比圆圈复杂些，可能有4个交点，3个交点，2个交点，1个交点，甚至于都不相交。由于圆圈和线段的长度同为 πd ，根据机会均等的原理，当它们均投掷 n 次(n 是一较大的数)，两者与平行线组交点的总数期望值也是一样的，即均为 $2n$ 。
- 长度为 s 的线段，投掷 n 次后，交点的个数为记为 m ，那么：

投针实验 $s : n : m$

圆圈拉直实验 $\pi d : n : 2n$

因此有： $m/2n = s/\pi d$

$$\pi = 2ns/md$$



蒙特卡罗方法

- 布丰投针实验是第一个用几何形式表达概率问题的例子，他首次使用随机实验处理确定性数学问题，为概率论的发展起到一定的推动作用。
- 像投针实验一样，用通过概率实验所求的概率来估计我们感兴趣的一个量，这样的方法称为**蒙特卡罗方法**（Monte Carlo method）。蒙特卡罗方法是在第二次世界大战期间随着计算机的诞生而兴起和发展起来的。这种方法在应用物理、原子能、固体物理、化学、生态学、社会学以及经济行为等领域中得到广泛利用。
- 蒙特·卡罗方法（Monte Carlo method），也称统计模拟方法，是二十世纪四十年代中期由于科学技术的发展和电子计算机的发明，而被提出的一种以概率统计理论为指导的一类非常重要的数值计算方法。是指使用随机数（或更常见的伪随机数）来解决很多计算问题的方法。蒙特·卡罗方法的名字来源于摩纳哥的一个城市蒙地卡罗，该城市以赌博业闻名，而蒙特·卡罗方法正是以概率为基础的方法。



随机非重复采样问题

- 设有 n 个样本，要求从中随机选出 m 个样本($m < n/2$)。请设计一个随机算法来完成该任务，并分析该算法的时间复杂度。
- 思路：
 - 将 n 个样本存放于数组 $\text{Sample}[1 \dots n]$ 中。数组 $\text{Result}[1 \dots m]$ 存放选中的 m 个样本。
 - 定义一个长度为 n 的标志数组 $\text{Flag}[1 \dots n]$ 。 $\text{Flag}[i] = \text{true}$ 表示对应位置的样本已经被选中；否则为未选中。
 - 随机生成一个落在区间 $[1, n]$ 的随机数 r 。若 $\text{Flag}[r]$ 为 false ， $\text{Sample}[r]$ 追加到数组 Result 中，并将 $\text{Flag}[r]$ 置为 true ；若 $\text{Flag}[r]$ 为 true ，则重新生成随机数 r ，直到 $\text{Flag}[r]$ 为 false 。重复此步骤，直到 m 个样本选择完成。

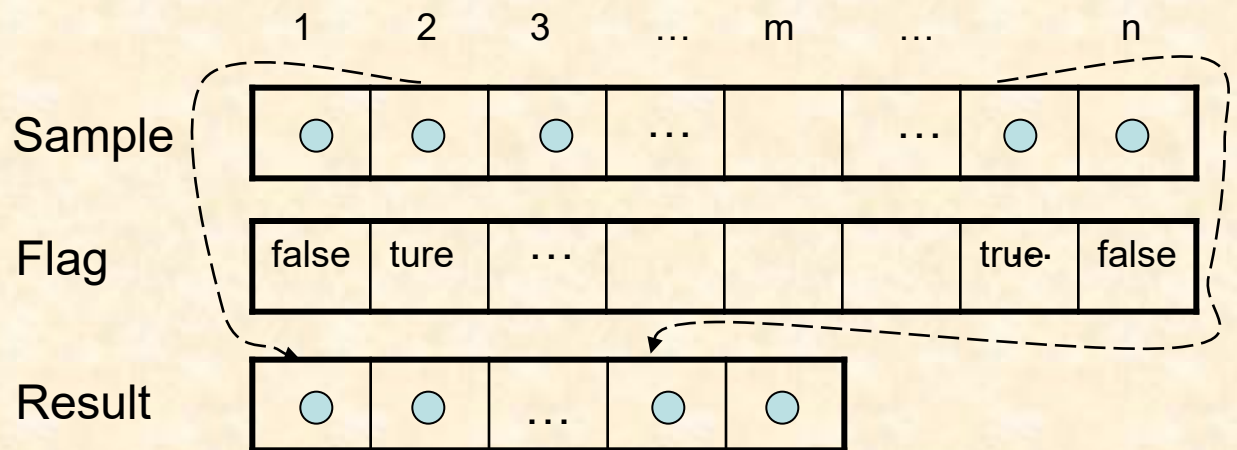


南京理工大学

输入：样本数组Sample[1...n], 选择样本的个数m, 且 $m < n$

输出：选中的样本Result[1..m]

```
1. for i ← 1 to n
2.   Flag[i] ← false //boolean数组，表示对应的样本是否已被选中
3. end for
4. k ← 0
5. while k < m
6.   r ← random(1, n)
7.   if not Flag[r] then
8.     k ← k + 1
9.     Result[k] ← Sample[r]
10.    Flag[r] ← true
11.  end if
12. end while
13. return Result[1...m]
```





时间复杂度分析

- 很显然，这是一个典型的迭代算法，因而可以使用计算迭代次数的技术来分析。
- 观察：然而每次运行此算法，迭代次数都可能是不同的。



- 引理1：在某个空间中抛掷硬币，设正面朝上的概率是 p ，反面朝上的概率为 $q(q=1-p)$ 。用随机变量 X 表示“第一次出现正面朝上”需要连续抛掷的次数，则随机变量 X 的分布满足几何分布

$$\mathbf{Pr}(X = k) = \begin{cases} pq^{k-1} & \text{if } k \geq 1 \\ 0 & \text{if } k < 1 \end{cases}$$

X 的数学期望是

$$\mathbf{E}(X) = \sum_{k=1}^{\infty} k \cdot \mathbf{Pr}(X = k) = \sum_{k=1}^{\infty} kpq^{k-1} = \frac{1}{p}$$

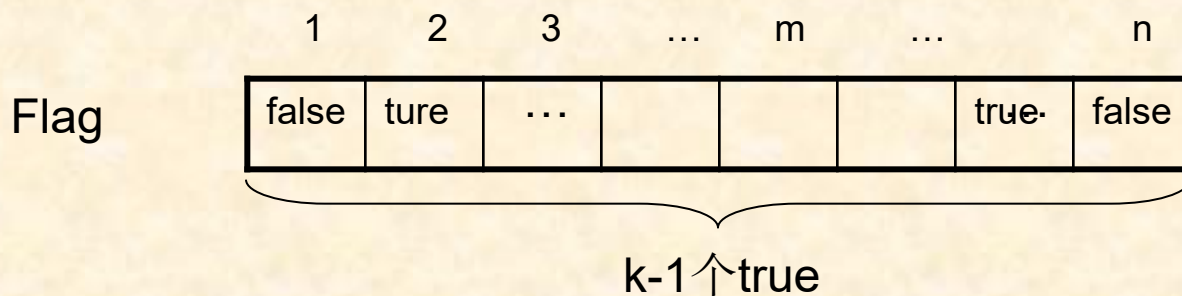
注：

$$\begin{aligned} \mathbf{E}(X) &= \sum_{k=1}^{\infty} kpq^{k-1} = p \sum_{k=1}^{\infty} kq^{k-1} = p \sum_{k=1}^{\infty} \frac{d}{dq} q^k = p \frac{d}{dq} \sum_{k=1}^{\infty} q^k \\ &= p \frac{d}{dq} \left(\frac{q}{1-q} \right) = p \frac{1}{(1-q)^2} = \frac{1}{p} \end{aligned}$$

$\mathbf{E}(X)$ 的含义是：平均连续抛掷 $1/p$ 次，会出现一次正面。



- 引理2：设随机数 r 在 $[1, n]$ 内满足均匀分布，则当已经选定 $k-1$ 个数时，随机函数一次就生成一个未被选定的整数的概率 p_k （即产生一个有效的随机数的概率 p_k ）为 $(n-k+1)/n$



“空位置个数”： $n-(k-1)$

“全部位置个数”： n

“随机数 r 落在空位置的概率”： $(n-(k-1))/n$



由引理 1 与引理 2 做如下类比：

	抛硬币	执行随机函数
感兴趣的事件	硬币正面朝上	产生一个有效的随机数
一次随机试验	概率为 p	(第 k 个随机数) 概率为 p_k
多次随机试验	硬币正面朝上第一次 出现时总的抛掷次数 X 满足几何分布	有效的随机数第一次出现时 随机函数的执行次数 X_k 满 足几何分布 $\Pr(X_k=t)$
数学期望	$E(X) = 1/p$	$E(X_k) = 1/p_k \ (1 \leq k \leq m)$

其中

$$\Pr(X_k = t) = \begin{cases} p_k (1 - p_k)^{t-1} & \text{if } t \geq 1 \\ 0 & \text{if } t < 1 \end{cases}$$

$$E(X_k) = \sum_{t=1}^{\infty} t \cdot \Pr(X_k = t) = \frac{1}{p_k} = \frac{n}{n - k + 1} \quad (1 \leq k \leq m)$$



- 用随机变量Y表示为了从n个样本中选出m个样本而生成的总的随机数个数(即算法总的迭代次数)，根据数学期望的线性性质，我们有

$$\begin{aligned} E(Y) &= E(X_1) + E(X_2) + \dots + E(X_m) \\ &= \sum_{k=1}^m E(X_k) \\ &= \sum_{k=1}^m \frac{n}{n-k+1} \\ &= n \sum_{k=1}^n \frac{1}{n-k+1} - n \sum_{k=m+1}^n \frac{1}{n-k+1} \\ &= n \sum_{k=1}^n \frac{1}{k} - n \sum_{k=1}^{n-m} \frac{1}{k} \end{aligned}$$



由于： $\sum_{k=1}^n \frac{1}{k} \leq \ln n + 1, \quad \sum_{k=1}^{n-m} \frac{1}{k} \geq \ln(n - m + 1)$

则有：
$$\begin{aligned} E(Y) &= n \sum_{k=1}^{\mathbf{n}} \frac{1}{k} - n \sum_{k=1}^{\mathbf{n-m}} \frac{1}{k} \\ &\leq n(\ln n + 1) - n \ln(n - \mathbf{m} + \mathbf{1}) \\ &\leq n(\ln n + 1) - n \ln(n - m) \\ &\leq n(\ln n + 1) - n \ln(n - \mathbf{n/2}) \quad (\because \mathbf{m} \leq \mathbf{n/2}) \\ &= n(\ln n + 1 - \ln(n / 2)) \\ &= n(\ln 2 + 1) \\ &\approx 1.69n \end{aligned}$$

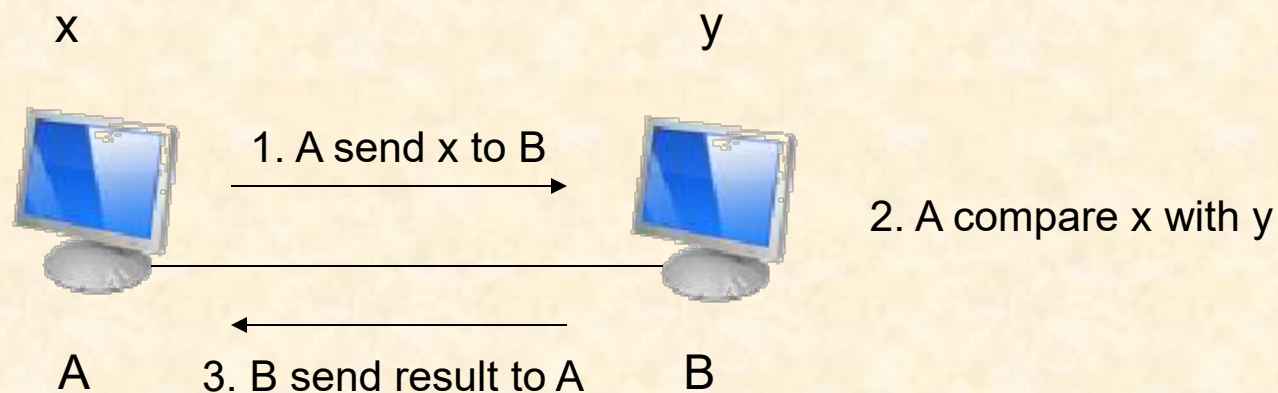
算法的期望运行时间 $T(n)$ 主要有两部分组成： 1)将boolean数组 $S[1 \dots n]$ 置为false耗时 $\Theta(n)$ ， 2)产生随机数 $r \leftarrow \text{random}(1, n)$ 的期望运行时间 $E(Y) \approx 1.69n$ ； 因此， 期望运行时间

$$T(n) \approx \Theta(n) + 1.69n = \Theta(n)$$

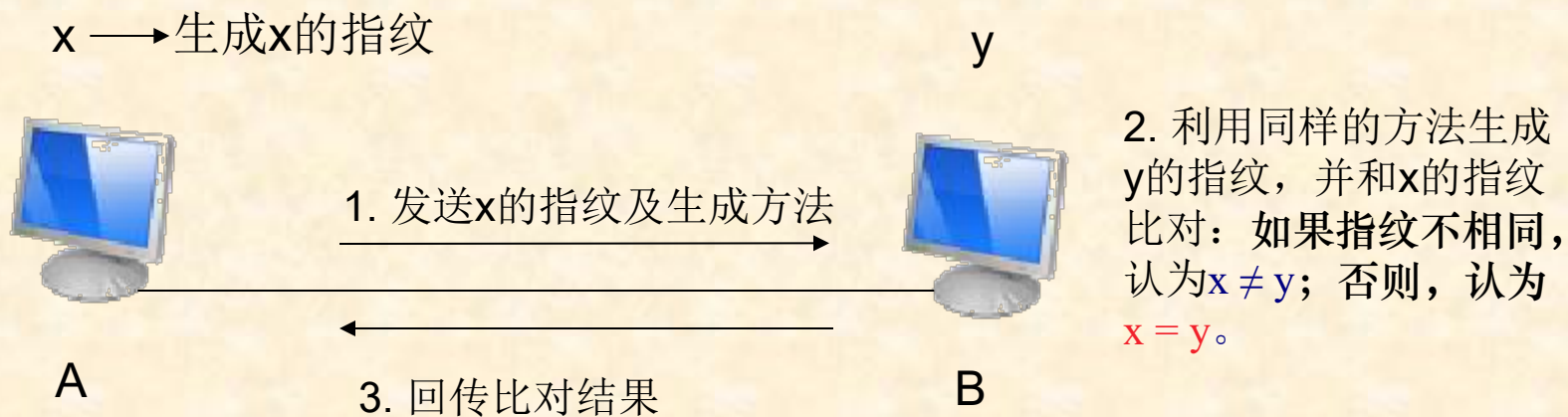
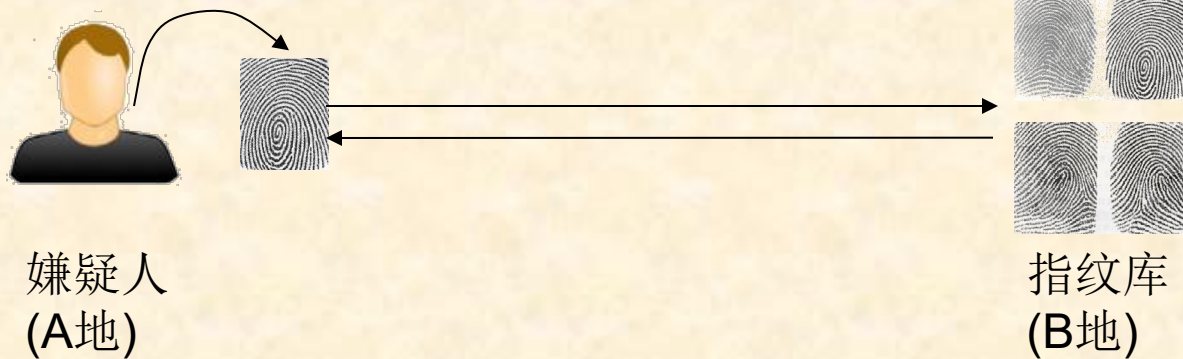


测试串的相等性

- 通信问题：发射端A发送信号 x （ x 一般为很长的二进制串），接收端B收到信号 y ，要确定是否 $x = y$ 。



目标：降低通信量，以减少对通信资源的占用。





生成指纹

对于一个 n 位(bit)的二进制串 x ， $I(x)$ 为该二进制串所表示的一个整数。一种生成指纹的方法是：选择一个素数 p ，令

$$I_p(x) = I(x) \pmod{p}$$

$I_p(x)$ 即作为 x 的指纹。

因为 $I_p(x) < p$ ，则 $I_p(x)$ 可用一个不超过 $\lfloor \log p \rfloor + 1$ 位的二进制串来表示。因此，如果 p 不是很大，那么，指纹 $I_p(x)$ 可以作为一个较短的串来发送，串长度为 $O(\log p)$ 。

例如： $x = (101011)_2 = (43)_{10}$ ，取 $p = (101)_2 = (5)_{10}$ ，
则 $I(x) \pmod{p} = 43 \pmod{5} = 3 = (11)_2$ ，仅需2比特。



分析

- 上述算法如果给出 $x \neq y$ ，肯定正确；如果给出 $x = y$ ，则可能出错。
- 出现错误匹配情形是：

$$x \neq y, \text{ 但 } I_p(x) = I_p(y)$$



$$x \neq y, \text{ } p \text{ 整除 } I(x) - I(y)$$

例： $x=(101011)_2=(43)_{10}$, $y=(110101)_2=(53)_{10}$ 。若取 $p=(101)_2=(5)_{10}$ ，则会出现错误匹配，即：

$$x \neq y, \text{ 但 } I_p(x) = 3 = I_p(y).$$

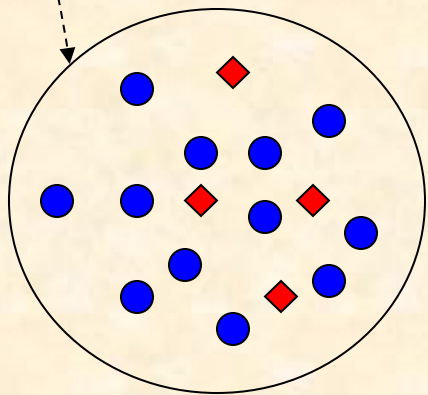


下面我们先给出算法的框架，再分析出错的概率。

算法：

1. 从小于M的素数集中随机选择一个 p
2. A 将 p 和 $I_p(x)$ 发送给B
3. B 检查是否 $I_p(x) = I_p(y)$ ，从而确定 x 是否和 y 相等。

当算法报告 $x=y$ 时，可能会出错。是否会出错取决于所选择的素数 p 。因此，该算法的出错概率为：



$$P_{error} = \frac{\text{会导致出错的素数个数(红色)}}{\text{候选素数个数(红色 + 蓝色)}}$$



1. 用 $\pi(n)$ 表示小于整数 n 的不同素数的个数。那么 $\pi(n)$ 渐趋近于 $n/\ln(n)$ 。
2. 如果整数 $k < 2^n$ ，那么能够整除 k 的素数的个数小于 $\pi(n)$ 。

$$P_{error} = \frac{\text{会导致出错的素数个数}}{\text{候选素数个数}} = \frac{|\{p \mid x \neq y, p \text{ 整除 } I(x) - I(y)\}|}{\pi(M)}$$

由于， x, y 均为 n 位二进制串，所以：

$$0 < I(x), I(y) < 2^n, \quad -2^n < I(x) - I(y) < 2^n.$$

则由结论2可知，整除 $I(x) - I(y)$ 的素数个数小于 $\pi(n)$ 。

$$P_{error} \leq \frac{\pi(n)}{\pi(M)}$$

若取 $M = 2n^2$ ，则：

$$P_{error} \leq \frac{\pi(n)}{\pi(M)} \approx (n / \ln n) / (2n^2 / \ln 2n^2) = \frac{n}{\ln n} \times \frac{\ln 2 + 2 \ln n}{2n^2} \approx \frac{1}{n}$$

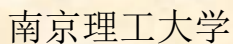


连续执行k次，则出错概率可以降低为：

$$P_{error} \leq \left(\frac{1}{n}\right)^k$$

例子：传输一个百万位的串即 $n=1,000,000$ ，取 $M=2n^2=2 \times 10^{12}$ ，传输素数 p 至多需 $\lfloor \log(M) \rfloor + 1 = 41$ 位，传输指纹也至多41位，共82位。验证1次发生假匹配的概率为 $1/1,000,000$ ，重复验证5次，假匹配概率可减小到 $(10^{-6})^5 = 10^{-30}$ 。

例： $x=(101011)_2=(43)_{10}$, $y=(110101)_2=(53)_{10}$ ，
取 $p=(101)_2=(5)_{10}$, $I_p(x)=3=I_p(y)$ ，出现假匹配；
再取 $p=3$ ， $I_p(x)=1 \neq I_p(y)=2$ ，验证不通过；



- 问题描述：给一个字模式串 $X=x_1x_2\dots x_n$ ，模式串 $Y=y_1y_2\dots y_m$ ，其中 $m \leq n$ ，问：模式串 Y 是否在模式串 X 中出现？不失一般性，假设模式串定义在字符集 $\{0,1\}$ 上。
- 最直观的方法：沿模式串 X 滑动 Y ，逐个比较子模式串 $X(j)=x_j\dots x_{j+m-1}$ 和 Y 。时间复杂度：最坏情况下为 $O(mn)$ ，例如：

$$Y = '000001'$$
$$\text{共}m^*(n-m)=O(mn)$$

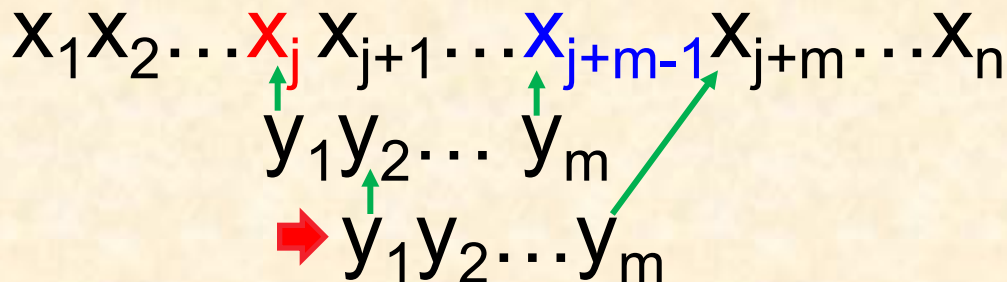


随机算法

- 思想：同样沿着模式串 X 滑动 Y ，但不是直接将模式串 Y 与每个子模式串 $X(j)=x_j \dots x_{j+m-1}$ 进行比较；而是借鉴指纹匹配的思想，将 Y 的指纹与子模式串 $X(j)$ 的指纹比较，从而判断 Y 是否与 $X(j)$ 相同。
- 直接使用上述方法时间复杂性不能有效降低，因为指纹计算同样要耗费时间。
- 然而，分析可以发现：新串 $X(j+1)$ 的指纹可以很方便地从 $X(j)$ 的指纹计算出来，即：

$$I_p(X(j+1)) = (2I_p(X(j)) - 2^m x_j + x_{j+m}) \pmod{p}$$

- 为何有上述结论？





$$\begin{array}{c}
 X_1 X_2 \cdots \color{red}{X_j} X_{j+1} \cdots \color{blue}{X_{j+m-1}} X_{j+m} \cdots X_n \\
 \quad \quad \quad \uparrow \quad \quad \quad \uparrow \quad \quad \quad \nearrow \\
 \quad \quad \quad y_1 y_2 \cdots y_m \\
 \quad \quad \quad \uparrow \\
 \color{red}{\rightarrow} y_1 y_2 \cdots y_m
 \end{array}$$

$$I(X(j)) = \color{red}{x_j} 2^{m-1} + x_{j+1} 2^{m-2} + x_{j+2} 2^{m-3} + \cdots + x_{j+m-1} 2^0 \quad (1)$$

$$I(X(j+1)) = x_{j+1} 2^{m-1} + x_{j+2} 2^{m-2} + \cdots + x_{j+m-1} 2^1 + \color{red}{x_{j+m}} 2^0 \quad (2)$$

$$2 \times (1): \quad 2I(X(j)) = x_j 2^m + x_{j+1} 2^{m-1} + \cdots + x_{j+m-1} 2^1 \quad (3)$$

$$(3) - (1): \quad 2I(X(j)) - I(X(j+1)) = x_j 2^m - x_{j+m} 2^0$$



$$I(X(j+1)) = 2I(X(j)) - x_j 2^m + x_{j+m} 2^0$$



$$I_p(X(j+1)) = (2I_p(X(j)) - 2^m x_j + x_{j+m}) \pmod{p}$$

注意，这里

$$\begin{aligned}
 (a \pm b) \pmod{p} &= \color{red}{a} \pmod{p} \pm \color{red}{b} \pmod{p} = \color{red}{c} \pm \color{red}{b} \pmod{p} \\
 &= \color{red}{c} \pmod{p} \pm \color{red}{b} \pmod{p} = (\color{red}{c} \pm \color{red}{b}) \pmod{p} \\
 &= (I_p(\color{red}{a}) \pm \color{red}{b}) \pmod{p}
 \end{aligned}$$



输入：模式串X，长度为n，模式串Y，长度为m

输出：若Y在X中出现，则返回Y在X中的第1个位置，否则返回-1

1. 从小于M的素数集中随机选择一个素数p
2. $j \leftarrow 1$
3. 计算 $W_p = 2^m \pmod p$, $I_p(Y)$ 和 $I_p(X_1)$ // $O(m) + O(m) + O(m)$
4. while $j \leq n - m + 1$ // $X_j = x_j \dots x_{j+m-1}$, 须有 $j + m - 1 \leq n$, 即 $j \leq n - m + 1$
 5. if $I_p(X_j) = I_p(Y)$ then return j // (可能)找到了匹配子串, $O(\log p)$
 6. $I_p(X_{j+1}) \leftarrow (2I_p(X_j) - x_j W_p + x_{j+m})$ // $O(1) * (n - m + 1) = O(n)$
 7. $j \leftarrow j + 1$
8. end while
9. return -1 // Y肯定不在X中(若在, 已由第5步返回j值)

时间复杂度分析: $O(m) + O(n) + O(\log p) = O(m+n)$



分析算法失败的可能性:

5. if $I_p(X_j) = I_p(Y)$ then return j //(可能)找到了匹配子串

出现错误匹配 $\Leftrightarrow Y \neq X(j)$ 但 $I_p(X_j) = I_p(Y)$ 。

即: 对于某个位置的 j , 存在 p 整除 $I(Y) - I(X_j)$;

进一步, 如果被选的素数 p 整除

$$\prod_{\{j|Y \neq X(j)\}} |I(Y) - I(X(j))|$$

因为 Y, X_j 均为 m 位二进制数, $0 < I(Y), I(X_j) < 2^m, \Rightarrow I(Y) - I(X_j) < 2^m$

故有

$$\prod_{\{j|Y \neq X(j)\}} |I(Y) - I(X(j))| < (2^m)^{n-m+1} < (2^m)^n = 2^{mn}。$$

问题: 整除它的素数 p 有多少个?



1. 用 $\pi(n)$ 表示小于整数 n 的不同素数的个数。那么 $\pi(n)$ 渐趋近于 $n/\ln(n)$ 。
2. 如果整数 $k < 2^n$ ，那么能够整除 k 的素数的个数小于 $\pi(n)$ 。

$$P_{error} = \frac{\text{会导致出错的素数个数}}{\text{候选素数个数}}$$

$$\prod_{\{j|Y \neq X(j)\}} |I(Y) - I(X(j))| < 2^{mn}$$

$$= \frac{\text{能整除 } \prod_{\{j|Y \neq X(j)\}} |I(Y) - I(X(j))| \text{ 的素数个数}}{\text{候选素数个数}}$$

$$< \frac{\pi(mn)}{\pi(M)}$$



若选择 $M=2mn^2$, 则发生假匹配的概率

$$\begin{aligned} P_{error} &< \frac{\pi(mn)}{\pi(M)} \\ &= \frac{\pi(mn)}{\pi(2mn^2)} \approx \frac{mn / \ln(mn)}{2mn^2 / \ln(mn^2)} \\ &= \frac{\ln(mn^2)}{2n \ln(mn)} \\ &< \frac{\ln(mn)^2}{2n \ln(mn)} \\ &= \frac{1}{n} \end{aligned}$$



蒙特卡罗方法小结

- 蒙特卡洛型算法在一般情况下可以保证：对于问题的所有实例，都以高概率给出正确解，但是通常无法判断一个解是否正确。
- 蒙特卡洛型随机算法的特点是：总是给出解，但偶尔解是错的，不知道一个解是对是错。
- 然而，可以多次运行原算法，设法使得每次运行时的随机选择都相对独立，则可以使非正确解的概率可以减小到任意小。



拉斯维加斯(Las Vegas)型随机算法

- 拉斯维加斯算法特点是“**或者给出正确解**”、“**或者无解**”。该类型算法不时做出可导致算法陷入僵局的选择，并且算法能够检测是否陷入僵局，如果是，算法承认失败。但是，只要这种行为出现的概率不占多数，**当出现失败时，只要在相同的输入实例上再次运行随机算法，就又有成功的可能。**
- 拉斯维加斯算法中的随机性选择能够引导算法快速求解问题，显著地改进算法的时间复杂性，甚至对某些迄今为止找不到有效算法的问题，也能找到满意的解。



高斯8皇后问题

该问题的解可表示为 $(x_1, \dots, x_8)$ ，其中 x_i 表示第 i 个皇后放置在第 i 行的第 x_i 列。约束条件： $x_i \neq x_j$ 且 $|x_i - x_j| \neq |i - j|$ 。

思路：在棋盘上相继的各行随机放置皇后，使新放置的皇后与已放置的皇后互不攻击，直到 8 个皇后均放好，或下一个皇后没有可放置的位置。



1. for $i \leftarrow 1$ to 8
2. $x[i] \leftarrow 0$
3. end for
4. count $\leftarrow 0$ //试探次数
5. for $i \leftarrow 1$ to 8
6. $j \leftarrow \text{random}(1,8)$
7. count $\leftarrow \text{count} + 1$
8. if 皇后 i 在位置 j 不冲突 then
9. $x[i] \leftarrow j$; count $\leftarrow 0$; $i \leftarrow i + 1$; 转步骤5
10. else if count = 8 then output “*failure*”
11. else 转步骤6 //重新放置皇后
12. end if
13. end for
14. return $x[1 \dots 8]$

回顾： n 个数，随机取 m 个，算法复杂度？



- 反复调用**Las Vegas**型8皇后问题的随机算法，直到得到问题的一个解。
- 将随机性算法与回溯法相结合，会获得更好的效果。可在棋盘上若干行随机放置相容的皇后，然后在其他行用回溯法继续放置，直到找到一个解或宣布算法失败。
- 特点：随机放置的皇后越多，回溯法搜索所需时间越少，但失败的概率越大。



将串匹配问题的Monte Carlo算法转换为Las Vegas算法

- Las Vegas算法特点：要么找到正确解，要么算法失败。
- Las Vegas算法与Monte Carlo算法的交集？
- 策略：只要产生匹配，就测试模式串Y与子串 X_j 是否相等，以确保找到正确解。

4. while $j \leq n-m+1$

5. if $I_p(X_j) = I_p(Y)$ then 测试Y与 X_j 是否相等

6. if $Y=X_j$ then return j

7. end if //若产生假匹配，则继续滑动模式串，进行下一次比较

7. $I_p(X_{j+1}) \leftarrow (2I_p(X_j) - x_j W_p + x_{j+m})$

8. $j \leftarrow j+1$

9. end while

10. return 0 // Y肯定不在X中(若在，已由第6步返回j值)



Las Vegas算法时间期望复杂度

$$O(n+m) \times (1-1/n) + O(mn) \times (1/n) = O(n+m)$$

其中：

- $1-1/n$ 为Monte Carlo算法成功的概率，
- $1/n$ 为调用Las Vegas算法的概率，
- $O(mn)$ 为发生假匹配时，用于“比较Y与 X_j 是否相等”操作的最坏时间复杂度。(最坏情况：每次都发生假匹配,则有： $1 \leq j \leq n-m+1$,每次耗时 $O(m)$ ，共 $O(mn)$)

至此，找到了一个总是有效的模式匹配算法，复杂度为 $O(n+m)$ 。



舍伍德(Sherwood)型随机算法

- 很多算法对不同的输入实例，运行时间差别很大。例如快速排序算法。此时，可通过舍伍德型随机算法来消除算法的时间复杂性与输入实例之间的关系。

例：

快速排序问题： 6, 13, 19, 23, 31, 35, 58

一次划分结果： 6, {13, 19, 23, 31, 35, 58}

随机选择主元： 23, 13, 19, 6, 31, 35, 58

一次划分结果： {6, 13, 19}, 23, {31, 35, 58}



随机化快速排序算法

输入：n个元素的数组 $A[1, \dots, n]$

输出：A中元素的非降序排列

random_quicksort(low, high)

1. if $low < high$ then
2. $v \leftarrow \text{random}(low, high)$
3. 互换 $A[low]$ 和 $A[v]$
4. split($A[low \dots high]$, w) //w是主元的新位置
5. random_quicksort(low, w-1)
6. random_quicksort(w+1, high)
- 7.end if

舍伍德型随机算法总能求得问题的一个正确解，但其平均复杂性与最坏复杂性并没有改进。换言之，该型随机算法不能避免最坏情况，但能尽量消除输入实例对算法性能的影响。



寻找第k小元素的随机算法

选择第 k 小元素问题，核心在于先选择合适的划分基准。

用中位数作为划分基准可以保证在最坏情况下用线性时间完成选择第 k 小元素的任务，即

$$T(n) \leq T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + cn$$

可知： $T(n) \leq cn/(1-1/5-3/4)=20cn = \Theta(n)$

$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + 4n & \text{if } n \geq 44 \end{cases} \longrightarrow T(n) \leq \frac{4n}{1 - \frac{1}{5} - \frac{3}{4}} = 80n$$

常数因子太大



改进思路

- 1) 若简单地用待划分数组的第 1 个元素作为划分基准，则算法的平均性能较好，而在最坏情况下，则需要 $O(n^2)$ 的时间(?)。
- 2) 舍伍德随机算法随机选择一个数组元素作为划分基准，既能保证算法的线性时间平均性能，又避免计算中位数的麻烦。

？： $T(n)=T(n-1)+n$ 故有 $T(n)=O(n^2)$.



寻找第k小元素的（舍伍德型）随机算法

Algorithm: RandomSelect (A[low, high], k)

输入：数组A[low,...high]和整数k， $1 \leq k \leq \text{high}-\text{low}+1$

输出：A[low...high]中的第k小元素

1. $v \leftarrow \text{random}(\text{low}, \text{high})$

2. $x \leftarrow A[v]$

3. 将A[low...high]分成三组

$$A_1 = \{a | a < x\} \quad A_2 = \{a | a = x\} \quad A_3 = \{a | a > x\} \quad // \Theta(n)$$

4. case

$|A_1| \geq k$: return RandomSelect($A_1[1, |A_1|]$, k)

$|A_1| + |A_2| \geq k$: return x

$|A_1| + |A_2| < k$: return RandomSelect($A_3[1, |A_3|]$, $k - |A_1| - |A_2|$)

5. end case



时间复杂度分析

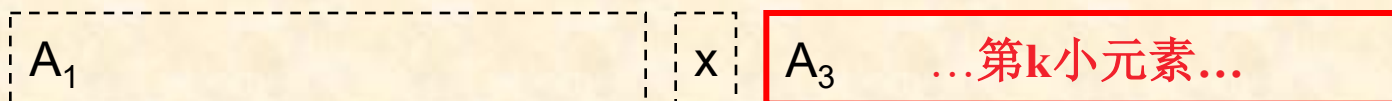
4. case

$|A_1| \geq k$: return RandomSelect($A_1[1, |A_1|]$, k)

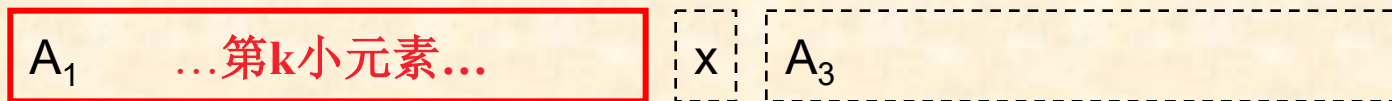
$|A_1| + |A_2| \geq k$: return x

$|A_1| + |A_2| < k$: return RandomSelect($A_3[1, |A_3|]$, $k - |A_1| - |A_2|$)

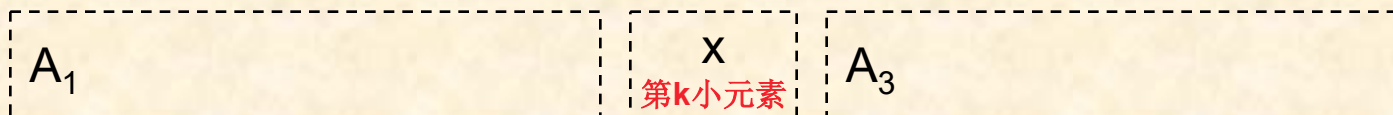
不妨设基准元素 x 为第 i 小元素，划分以后， x 与目标值（即第 k 小元素）之间不外乎以下3种情况：



1) $i < k$, 递归搜索 A_3 , 剩余 $n-i$ 个元素, 如上图



2) $i > k$, 递归搜索 A_1 , 剩余 $i-1$ 个元素, 如上图

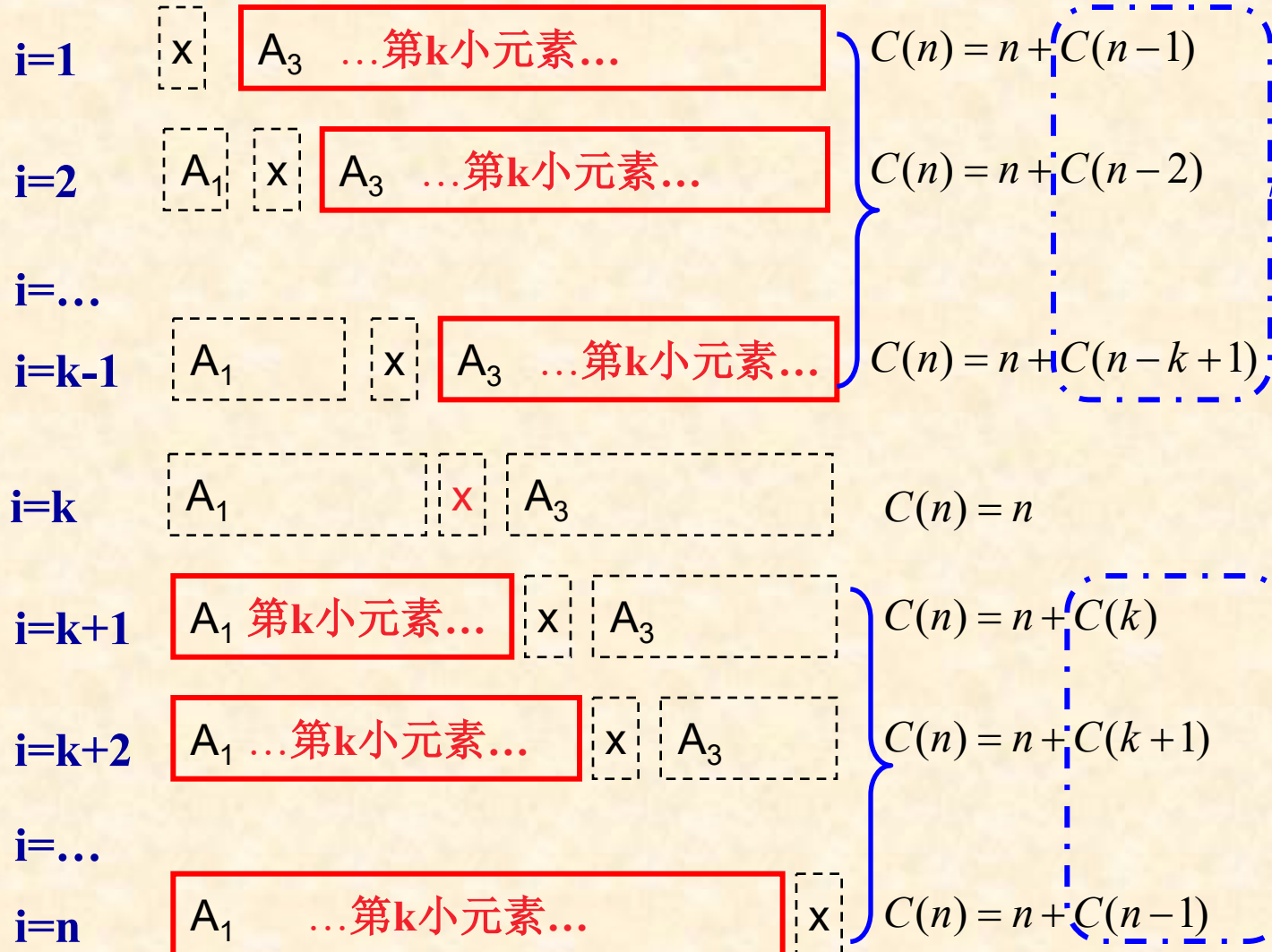


3) $i = k$, 找到目标了! 算法结束



$$C(n) = n + \frac{1}{n} \left[\sum_{i=1}^{k-1} C(n-i) + \sum_{i=k+1}^n C(i-1) \right]$$

时间复杂度分析





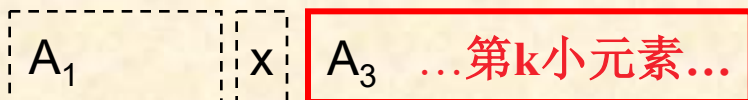
时间复杂度分析

- 假设元素x的位置(i)以相等概率分布于1,2,...,n，则算法的期望比较次数为

$$C(n) = n + \frac{1}{n} \left[\sum_{i=1}^{k-1} C(n-i) + \sum_{i=k+1}^n C(i-1) \right]$$

$i < k$, 递归搜索 A_3 ,
剩余 $n-i$ 个元素

$i > k$, 递归搜索 A_1 ,
剩余 $i-1$ 个元素





时间复杂度分析

下面用第二数学归纳法证明： $C(n) \leq 4n$

易验证： $n=2$ 或 3 时， $C(2) \leq 4 \times 2 = 8$ ， $C(3) \leq 4 \times 3 = 12$ ，假设成立。

设对所有的 k ， $1 \leq k \leq n-1$ ， $C(k) \leq 4k$ 成立，需证明 $C(n) \leq 4n$ 成立。

$$\begin{aligned} C(n) &= n + \frac{1}{n} \left[\sum_{i=1}^{k-1} C(n-i) + \sum_{i=k+1}^n C(i-1) \right] \\ &= n + \frac{1}{n} \left[\sum_{j=n-k+1}^{n-1} C(j) + \sum_{j=k}^{n-1} C(j) \right] \\ &\leq n + \frac{1}{n} \max_k \left[\sum_{j=n-k+1}^{n-1} C(j) + \sum_{j=k}^{n-1} C(j) \right] \end{aligned}$$

因 $C(j)$ 为 j 的非递减函数，故上表达式在 $k = \lceil n/2 \rceil$ 时最大。



命题： $f(k) = \sum_{j=n-k+1}^{n-1} C(j) + \sum_{j=k}^{n-1} C(j)$ 在 $k=\lceil n/2 \rceil$ 时最大

证明思路： 1) 先证 $k < \lfloor n/2 \rfloor$ 时，函数递增
 2) 再证 $k > \lceil n/2 \rceil$ 时，函数递减.
 3) 讨论 $k = \lfloor n/2 \rfloor, \lceil n/2 \rceil$ 时的增减性

证明： 设有 $k < k' < \lfloor n/2 \rfloor$

$$f(k) = \underbrace{C(k) + \dots}_{k'-k} + C(k') + \dots + C(n-1) \square + C(n-1) \square + \dots + C(n-k+1)$$

$$f(k') = C(k') + \dots + C(n-1) \square + C(n-1) \square + \dots + C(n-k+1) \overbrace{+ C(n-k) + \dots + C(n-k'+1)}^{k'-k}$$

$$\begin{aligned} f(k') - f(k) &= C(n-k'+1) - C(k) \quad // \quad n-k'+1-k = n+1-(k+k') \\ &\quad + C(n-k'+2) - C(k+1) // \quad n-k'+2-(k+1) = n+1-(k+k') \\ &\quad + \dots \\ &\quad + C(n-k) - C(k'-1) \quad // \quad n-k-(k'-1) = n+1-(k+k') \end{aligned}$$



自变量之差 $n+1-(k+k')$ 所揭示的单调性

讨论：1) 当 $k < k' < \lfloor n/2 \rfloor$ 时， $k+k' < 2 * \lfloor n/2 \rfloor = n$ (或 $n-1$)，则
 $n+1-(k+k') > 0$ ，因 $C(j)$ 单调递增，则 $f(k')-f(k)$ 的每一项均 >0 ，因此 $f(k') > f(k)$ ，即 $f(k)$ 单调递增。

2) 当 $\lceil n/2 \rceil < k < k'$ 时，

若 n 为偶数，有 $k > \lceil n/2 \rceil = n/2$ ， $k' \geq n/2 + 1$ ，则 $k+k' > n+1$ ，
即 $n+1-(k+k') < 0$ 。

若 n 为奇数，有 $k > \lceil n/2 \rceil = (n+1)/2$ ， $k' \geq (n+1)/2 + 1$ ，则有
 $k+k' > 2 * (n+1)/2 + 1 = n+2$ ，则 $n+1-(k+k') < 0$ 。

因此，当 $\lceil n/2 \rceil < k < k'$ 时， $n+1-(k+k') < 0$ ，

因 $C(j)$ 单调递增，则 $f(k')-f(k)$ 的每一项均 <0 ，

故 $f(k') < f(k)$ ，即 $f(k)$ 单调递减。

综上，(特别地，) n 为偶数时，当 $k = \lfloor n/2 \rfloor = \lceil n/2 \rceil = n/2$ ， $f(k)$ 取最大值。



讨论 $k=\lfloor n/2 \rfloor, \lceil n/2 \rceil$ 时 $f(k)$ 的增减性 (n 为奇数)

$$f(k) = \sum_{j=n-k+1}^{n-1} C(j) + \sum_{j=k}^{n-1} C(j)$$

$$\begin{aligned} f(\lfloor n/2 \rfloor) &= \sum_{j=n-\lfloor n/2 \rfloor+1}^{n-1} C(j) + \sum_{j=\lfloor n/2 \rfloor}^{n-1} C(j) = \sum_{j=\lceil n/2 \rceil+1}^{n-1} C(j) + \sum_{j=\lfloor n/2 \rfloor}^{n-1} C(j) \\ &= 2 \sum_{j=\lceil n/2 \rceil+1}^{n-1} C(j) + C(\lceil n/2 \rceil) + \mathbf{C}(\lfloor n/2 \rfloor) \end{aligned}$$

$$\begin{aligned} f(\lceil n/2 \rceil) &= \sum_{j=n-\lceil n/2 \rceil+1}^{n-1} C(j) + \sum_{j=\lceil n/2 \rceil}^{n-1} C(j) = \sum_{j=\lfloor n/2 \rfloor+1}^{n-1} C(j) + \sum_{j=\lceil n/2 \rceil}^{n-1} C(j) = 2 \sum_{j=\lceil n/2 \rceil}^{n-1} C(j) \\ &= 2 \sum_{j=\lceil n/2 \rceil+1}^{n-1} C(j) + C(\lceil n/2 \rceil) + \mathbf{C}(\lceil n/2 \rceil) \end{aligned}$$

则 $f(\lceil n/2 \rceil) - f(\lfloor n/2 \rfloor) = \mathbf{C}(\lceil n/2 \rceil) - \mathbf{C}(\lfloor n/2 \rfloor) > 0$

即有 $\mathbf{f}(\lceil n/2 \rceil) > \mathbf{f}(\lfloor n/2 \rfloor)$ 成立。



n为奇数时, $f(k)$ 的单调性小结

$$f(k) = \sum_{j=n-\mathbf{k}+1}^{n-1} C(j) + \sum_{j=\mathbf{k}}^{n-1} C(j)$$

讨论: 1) 当 $k < k' < \lfloor n/2 \rfloor$ 时, $f(k)$ 单调递增。

2) 当 $\lceil n/2 \rceil < k < k'$ 时, $f(k)$ 单调递减。

3) 此外, $f(\lceil n/2 \rceil) > f(\lfloor n/2 \rfloor)$ 成立。

综上, (特别地,) n为奇数时, $f(k)$ 在 $k = \lceil n/2 \rceil$ 时取最大值。



时间复杂度分析

$$C(n) \leq n + \frac{1}{n} \left[\sum_{j=n-\textcolor{red}{k}+1}^{n-1} C(j) + \sum_{j=\textcolor{red}{k}}^{n-1} C(j) \right] \Bigg|_{k = \lceil n/2 \rceil}$$

$$= n + \frac{1}{n} \left[\sum_{j=n-\lceil n/2 \rceil+1}^{n-1} C(j) + \sum_{j=\lceil n/2 \rceil}^{n-1} C(j) \right]$$

$$\leq n + \frac{1}{n} \left[\sum_{j=\lfloor \textcolor{red}{n}/2 \rfloor+1}^{n-1} \textcolor{red}{4j} + \sum_{j=\lceil n/2 \rceil}^{n-1} \textcolor{red}{4j} \right] \quad // \text{因 } \lceil n/2 \rceil + \lfloor n/2 \rfloor = n$$

$$\leq n + \frac{4}{n} \left[\sum_{j=\lfloor \textcolor{red}{n}/2 \rfloor}^{n-1} j + \sum_{j=\lceil n/2 \rceil}^{n-1} j \right] \quad // \text{易验证 } \lfloor n/2 \rfloor + 1 \geq \lceil n/2 \rceil \text{ 成立}$$

$$\leq n + \frac{8}{n} \left[\sum_{j=\lfloor \textcolor{red}{n}/2 \rfloor}^{n-1} j \right]$$



时间复杂度分析

$$C(n) \leq n + \frac{8}{n} \left[\sum_{j=\lceil n/2 \rceil}^{n-1} j \right]$$

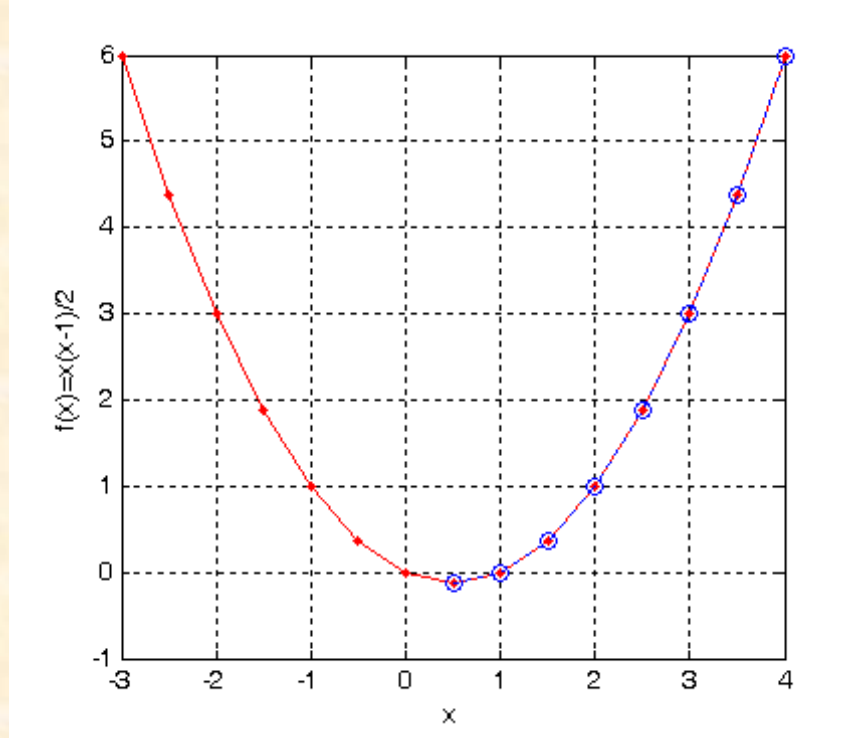
$$= n + \frac{8}{n} \left[\sum_{j=1}^{n-1} j - \sum_{j=1}^{\lceil n/2 \rceil - 1} j \right]$$

$$= n + \frac{8}{n} \left\{ \frac{n(n-1)}{2} - \frac{\lceil n/2 \rceil (\lceil n/2 \rceil - 1)}{2} \right\}$$

$$\leq n + \frac{8}{n} \left\{ \frac{n(n-1)}{2} - \frac{(n/2)(n/2-1)}{2} \right\}$$

$$= 4n - 2$$

$$< 4n$$



//易知 $\lceil n/2 \rceil \geq n/2 \geq 1/2$,

//函数 $f(x)=x(x-1)/2$ 在 $x \geq 1/2$ 时单调递增

//故 $\frac{\lceil n/2 \rceil (\lceil n/2 \rceil - 1)}{2} \geq \frac{(n/2)(n/2-1)}{2}$



时间复杂度分析

已证明: $C(n) \leq 4n$.

此外, 因每个元素至少与基准元素 x 比较1次, 故 $C(n) \geq n$.

因此, 期望元素比较次数为

$$n \leq C(n) \leq 4n$$

时间复杂度为 $\Theta(n)$

比较:

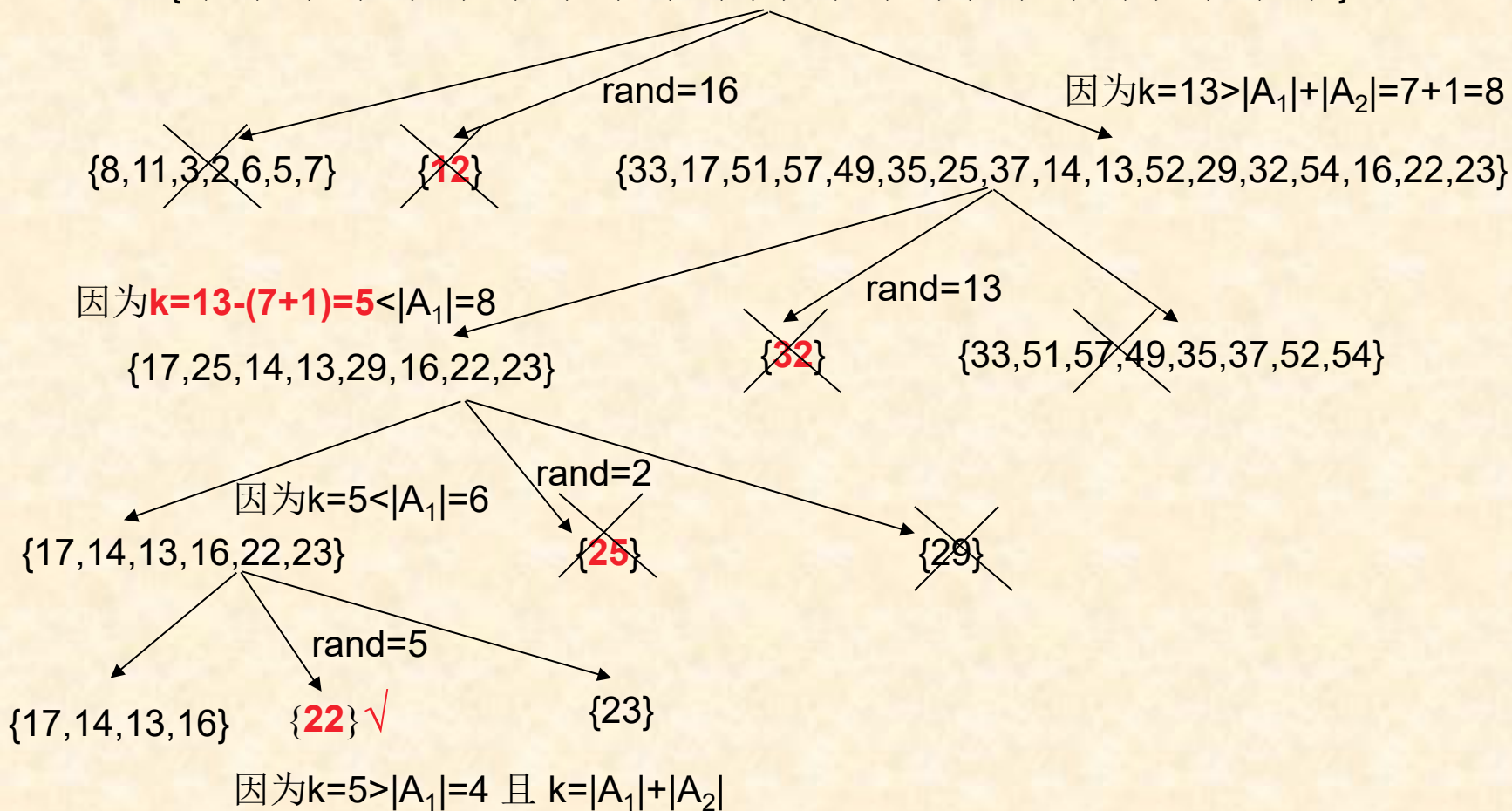
$$T(n) \leq \begin{cases} c & \text{if } n < 44 \\ T(\lfloor n/5 \rfloor) + T(\lfloor 3n/4 \rfloor) + 4n & \text{if } n \geq 44 \end{cases} \longrightarrow T(n) \leq \frac{4n}{1 - \frac{1}{5} - \frac{3}{4}} = 80n$$



例：线性随机查找

要寻找下面数组A中的第13小元素：

$A = \{8, 33, 17, 51, 57, 49, 35, 11, 25, 37, 14, 3, 2, 13, 52, 12, 6, 29, 32, 54, 5, 16, 22, 23, 7\}$





作业

- A: 14.7, 14.10, 14.16
- B: 编程实现14.4节的“随机化的选择算法”，并以表格形式给出若干组 $(n, T(n))$ 数据，其中, n 是问题规模, $T(n)$ 是元素比较次数；并绘图展示结果。